

LaRT AMR extension — 2026 session notes

LaRT development

2026 May

Contents

1	Overview	1
2	Architecture: RAMSES → generic → LaRT.x	1
3	Generic AMR file format	2
4	New shared physics module: <code>physics.amr_mod.f90</code>	2
5	New solar-abundance / CIE module: <code>ion_data_mod.f90</code>	3
6	New input parameters	3
7	Bug fixes (2026-05-21 session)	4
7.1	Bug 1: point source with <code>icell_amr</code> unset	4
7.2	Bug 2: <code>diffuse_emissivity</code> alias table not populated	4
7.3	Bug 3: pole traversal across octree gaps	4
7.4	Bug 4: ORIGINX/Y/Z coordinate convention	4
8	Python tools (<code>python/AMR_grid/</code>)	5
8.1	<code>convert_ramses_to_generic.py</code>	5
8.2	<code>extract_amr_subset.py</code> (new)	5
8.3	<code>AMR_grid.py</code> — in-memory AMR construction	5
8.4	<code>make_amr_grid.py</code> , <code>make_amr_sphere_radial.py</code>	6
9	End-to-end example: jellyfish	6
10	Validation summary	6

1 Overview

This document summarizes the AMR-side changes made to LaRT v2.10 (and mirrored in v2.00) across the 2026 sessions. It covers:

1. Converter-centric architecture (RAMSES direct read removed from `LaRT.x`).
2. Generic AMR file format extended with optional physics columns.
3. New shared physics module `physics.amr_mod.f90` (CIE neutral fraction, Case B Ly α emissivity, Laursen+09 dust).
4. New solar-abundance / CIE ion module `ion_data_mod.f90`.

5. New input parameters `par%ionization_model`, `par%dust_model`, `par%emissivity_model`, `par%ion_model`, `par%metallicity_global`, `par%Z_ref`, `par%f_ion_dust`.
6. Four bug fixes from the 2026-05-21 debugging session (*Bug 1–Bug 4*).
7. New Python tool `extract_amr_subset.py` for spatial sub-region extraction.
8. Centered coordinate convention applied uniformly to both converters (Python and Fortran).

2 Architecture: RAMSES $\rightarrow$ generic $\rightarrow$ LaRT.x

The previous architecture allowed LaRT.x to read RAMSES binary snapshots directly via `par%amr_type='ramses'`. The new architecture removes that path; instead a standalone converter produces a generic AMR file that LaRT.x consumes.

```
RAMSES snapshot --[convert_ramses_to_generic.x or .py]-->
generic AMR file (.h5 / .fits.gz / .dat) --> LaRT.x
```

Setting `par%amr_type='ramses'` now triggers a fatal error with a clear message pointing to the converter.

Why?

- Separates a one-off conversion step (RAMSES I/O, layout parsing, unit conversion) from the production RT loop.
- Makes adding new input simulation codes (AREPO, ATHENA, etc.) a pure converter problem; LaRT.x only knows the generic format.
- Lets physics computations (CIE, dust, Case B) be computed once at conversion time and stored alongside the cells, avoiding repeated recomputation per RT run.

3 Generic AMR file format

Mandatory columns (9). `x`, `y`, `z`, `level`, `nH` [cm^{-3}], `T` [K], `vx` [km/s], `vy` [km/s], `vz` [km/s].

Optional columns (6). Detected by column name in HDF5/FITS files; allocated only if present:

Name	Units	Used by
<code>metallicity</code>	mass fraction	dust model, ion density
<code>xHI</code>	dimensionless	ionization model 'from_file'
<code>n_e</code>	cm^{-3}	emissivity computation
<code>n_ion</code>	cm^{-3}	metal-line opacity
<code>emissivity</code>	$\text{cm}^{-3} \text{ s}^{-1}$	emissivity model 'from_file'
<code>ndust</code>	cm^{-3}	dust model 'from_file'

Coordinate convention. The file header carries `BOXLEN` and `ORIGINX/Y/Z`. LaRT now honors `ORIGINX/Y/Z`: the box is built as `[origin, origin + boxlen]` in the same frame as the input cell positions. The default in the absence of these headers is the centered convention

$[-\text{boxlen}/2, +\text{boxlen}/2]$.

Both converters (Python and Fortran) now write *centered* coordinates by default, i.e. they shift RAMSES native positions from $[0, \text{boxlen}]$ to $[-\text{boxlen}/2, +\text{boxlen}/2]$ and set `ORIGINX/Y/Z` = $-\text{boxlen}/2$. The previous corner-based convention (`ORIGINX` = 0) is still readable; LaRT will print a warning if the data does not cover the box center, since pole traversal starts at the box center.

4 New shared physics module: `physics_amr_mod.f90`

Used by both `LaRT.x` (in `grid_mod_amr.f90`) and the converter (in `convert_ramses_to_generic.f90`), so the formulas cannot drift between pre-computation and in-LaRT fallback.

```
public :: cie_neutral_fraction_formula  ! single CIE formula (legacy)
public :: cie_neutral_fraction_table    ! Voronov+Verner table
      interpolation
public :: laursen09_ndust                ! dust from metallicity
public :: caseB_lya_emissivity           ! recombination + collisional
public :: electron_density_from_xHI      !  $n_e \sim n_H(1 - x_{HI})$ 
```

CIE neutral-fraction table. Built once on first call from the Voronov (1997) collisional ionization rate and the Verner & Ferland (1996) Case A recombination rate:

$$x_{HI}(T) = \frac{\alpha_A(T)}{\Gamma_{HI}(T) + \alpha_A(T)},$$

with $\log_{10} T \in [3.0, 8.0]$ at $\Delta \log T = 0.1$, log-linear interpolation.

Case B Ly α emissivity.

$$\varepsilon_{Ly\alpha} = P_B \alpha_B n_e n_{HII} + q_{coll}(T) n_e n_{HI},$$

with α_B from Hui & Gnedin (1997), P_B from Cantalupo et al. (2008), and q_{coll} from the Harley/RASCAS fit.

Laursen+09 dust.

$$n_{\text{dust}} = \frac{Z}{Z_{\text{ref}}} (n_{HI} + f_{\text{ion}} n_{HII}),$$

where Z_{ref} and f_{ion} are exposed as `par%Z_ref` (default 0.0134) and `par%f_ion_dust` (default 0.01).

5 New solar-abundance / CIE module: `ion_data_mod.f90`

Provides $n_{\text{ion}}(nH, Z, T, \text{ion_id})$ for 11 ion species using Asplund+09 solar abundances and Gnat & Sternberg (2007) CIE ion fractions:

H I, He I, C IV, N V, O VI, Na I, Ca II, Mg II, Si IV, Si II, Al II, Fe II.

Activated by `par%ion_model = 'solar.cie'` when the input file does not carry an `n_ion` column.

6 New input parameters

Parameter	Default	Meaning / values
<code>par%ionization_model</code>	<code>'cie_formula'</code>	<code>'cie_formula'</code> (single Voronov formula), <code>'cie_table'</code> (Voronov + Verner table), <code>'full_neutral'</code> ($x_{HI} = 1$), <code>'from_file'</code> (requires <code>xHI</code> column).
<code>par%dust_model</code>	<code>'global_dgr'</code>	<code>'global_dgr'</code> (uses <code>par%DGR</code>), <code>'laursen09'</code> (metallicity-based), <code>'from_file'</code> (requires <code>ndust</code> column).
<code>par%emissivity_model</code>	<code>'none'</code>	<code>'none'</code> (use <code>par%emiss_file</code> or no diffuse source), <code>'caseB'</code> (recomb + collisional, per cell), <code>'from_file'</code> (requires <code>emissivity</code> column).
<code>par%ion_model</code>	<code>'none'</code>	<code>'none'</code> (use $n_{HI} = n_H \cdot x_{HI}$), <code>'solar_cie'</code> (Asplund+09 & Gnat-Sternberg+07), <code>'from_file'</code> (requires <code>n_ion</code> column).
<code>par%metallicity_global</code>	<code>-1</code>	Global Z used by <code>dust_model='laursen09'</code> or <code>ion_model='solar_cie'</code> when no <code>metallicity</code> column is present; negative $\Rightarrow$ unset.
<code>par%Z_ref</code>	<code>0.0134</code>	Reference solar metallicity for Laursen+09 dust scaling.
<code>par%f_ion_dust</code>	<code>0.01</code>	Survival fraction of dust in ionized gas (Laursen+09).

The defaults are chosen so legacy input files reproduce the previous behavior exactly.

7 Bug fixes (2026-05-21 session)

7.1 Bug 1: point source with `icell_amr` unset

When `par%source_geometry='point'`, `photon%icell_amr` was left at 0 by `generate_photon_amr`, while every other source-geometry branch set it. Subsequent ray-tracing then indexed the leaf arrays with 0 and crashed.

Fix: call `amr_find_leaf` after writing the point source coordinates, and stop with a clear error if the position is outside any leaf.

7.2 Bug 2: `diffuse_emissivity` alias table not populated

`setup.f90` promotes an unset `par%geometry` to `'sphere'`. In AMR mode this previously sent `diffuse_emissivity` photons through the 1D radial-profile path that uses `emiss_prof%prob_alias`, which is never populated by the AMR code. Result: `PROB_ALIAS` not associated (random_mt.f90:2301).

Fix: branch on `associated(emiss_prof%prob_alias)` rather than `par%geometry`. `amr_setup_emissivity` also refactored so it (a) returns early unless `source_geometry='diffuse_emissivity'`, (b) computes Case B emissivity from the real `caseB_lya_emissivity` formula rather than `rhokap`, (c) raises an explicit error if `from_file` is selected but no `emissivity` column is present, and (d) raises an error if the total cell emissivity is zero.

7.3 Bug 3: pole traversal across octree gaps

When the octree is incomplete (e.g. a spatial subset of a larger simulation), the pole traversal from box center along $+z$ may step into an internal cell whose subtree does not cover the photon position. `amr_next_leaf` returned 0, the traversal stopped, $N(HI)_{\text{pole}} = 0$, and `taumax`-normalization fell back to `tauhomo`, producing extreme cell opacities.

Fix: two new octree helpers — `amr_find_enclosing_cell` (returns the deepest cell containing a point, leaf or internal) and `amr_gap_exit` (steps through a virtual sub-cell of an internal cell). The pole traversal is rewritten as a single loop: at each step either accumulate opacity through a leaf, or step through a gap with zero opacity, until $z \geq z_{\text{max}}$.

Validated against an artificial pole-line gap in the AMR sphere example: $N(HI)_{\text{pole}}$ matches the original ungapped sphere exactly.

7.4 Bug 4: ORIGINX/Y/Z coordinate convention

`read_generic_amr.f90` previously assumed every input file was already in centered convention $[-\text{boxlen}/2, +\text{boxlen}/2]$ regardless of the FITS/HDF5 ORIGINX/Y/Z headers. Files written with corner-based positions (`ORIGINX = 0`, e.g. from the legacy jellyfish converter) ended up with an octree whose box was offset from the input data.

Fix: the reader now returns the origin recovered from the headers, defaulting to $-\text{boxlen}/2$ when the headers are missing. `grid_mod_amr.f90` calls `amr_build_tree` with `box = [origin, origin + boxlen]`, so the octree sits in the same frame as the input cells. Setup prints both the box extent and the actual data range, and warns if the data does not cover the box center.

8 Python tools (python/AMR_grid/)

8.1 convert_ramses_to_generic.py

Purpose. Convert a RAMSES snapshot to the generic AMR file format consumed by LaRT.x.

Output. Coordinates are written in *centered* convention; `ORIGINX/Y/Z = -boxlen/2`.

Usage.

```
# Basic conversion (cells + density + temperature + velocity)
python convert_ramses_to_generic.py <repo_dir> <snapnum> \
    -o sim.h5 --output-unit kpc

# With on-the-fly physics columns (CIE xHI, ne, Case B emissivity, dust)
python convert_ramses_to_generic.py <repo_dir> <snapnum> \
    -o sim.h5 --output-unit kpc \
    --compute-physics --metallicity 0.0134
```

Equivalent Fortran tool: `convert_ramses_to_generic.x` (built by `make convert_ramses`).

8.2 extract_amr_subset.py (new)

Purpose. Carve a cubic spatial sub-region out of a generic AMR file. Cells whose centres fall inside the cube are kept; coordinates are shifted into a centered output cube $[-\text{boxlen}_{\text{sub}}/2, +\text{boxlen}_{\text{sub}}/2]$.

Re-indexing. The output uses sequential row indices $1..N_{\text{subset}}$; the original RAMSES cell ID is not preserved (LaRT generic AMR files do not currently carry an explicit cell-ID column).

Usage.

```
# Explicit bounds (must be cubic)
python extract_amr_subset.py input.fits.gz \
    --xmin 0 --xmax 30 --ymin 0 --ymax 30 --zmin 0 --zmax 30 \
    -o sub.fits.gz

# Center + size
python extract_amr_subset.py input.fits.gz \
    --center 15 15 15 --size 30 \
    -o sub.fits.gz
```

Input: .fits, .fits.gz, or .h5. Output is inferred from the requested file extension.

8.3 AMR_grid.py — in-memory AMR construction

Unchanged from previous releases, but its default origin convention ($-\text{boxlen}/2$, centered) is now what the rest of the chain also produces. All optional physics columns (`metallicity`, `xHI`, `n_e`, `n_ion`, `emissivity`, `ndust`) are first-class `Cell` attributes and are written to FITS/HDF5 when set on any leaf.

8.4 make_amr_grid.py, make_amr_sphere_radial.py

Sphere and radial-profile generators. Both use the centered convention by default.

9 End-to-end example: jellyfish

`examples/jellyfish_rmhd/` now contains an end-to-end example with an analysis notebook. Workflow:

```
# 1) Start from the RAMSES-derived generic file (corner-based, BOXLEN=60)
ls jellyfish_sub.fits.gz

# 2) Extract a centered cube  $[0,30]^3 \rightarrow [-15,+15]^3$ 
python ../../python/AMR_grid/extract_amr_subset.py \
    jellyfish_sub.fits.gz \
    --center 15 15 15 --size 30 \
    -o jellyfish_sub_centered.fits.gz

# 3) Run LaRT
mpirun -np 70 ../../LaRT.x jellyfish_centered.in

# 4) Open the analysis notebook
jupyter notebook jellyfish_analysis.ipynb
```

The notebook plots the emergent spectrum $J_{\text{out}}(\nu)$, the 2D surface-brightness maps (Scattered / Direct / Total), and a position–velocity diagram from the data cube.

10 Validation summary

AMR sphere regression (cubic box, $[-1,+1]$). $N(HI)_{\text{pole}} = 1.6954 \times 10^{15}$ for all of: point source, `density1` diffuse emissivity, `density2` diffuse emissivity, `caseB` diffuse emissivity, and the artificial pole-line gap case.

Centered jellyfish subset. With `jellyfish_sub_centered.fits.gz` (`BOXLEN=30`, `ORIGINX=-15`): data x range $[-14.8, +15.0]$ fully covers the box center, $N(HI)_{\text{pole}} = 1.32 \times 10^{17}$, no warnings, photon RT runs to completion.

Corner-based jellyfish subset (legacy). With the original `jellyfish_sub.fits.gz` (`BOXLEN=60`, `ORIGINX=0`): data x range $[0.2, 30.0]$ does not cover the box center (30, 30, 30); LaRT emits a clear warning and proceeds with $N(HI)_{\text{pole}} = 0$, falling back to `tauhomo` normalization. The user is expected to either run `extract_amr_subset.py` or set `par%xs_point` inside the data region.